

Why is Redis fast?

1

All data is stored in memory

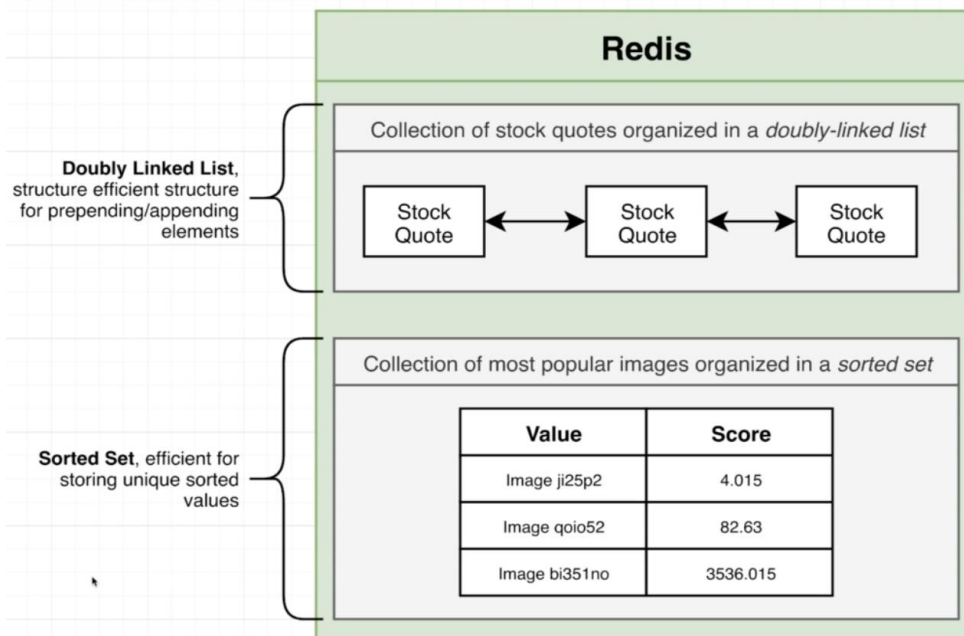
2

Data is organized in simple data structures

3

Redis has a simple feature set

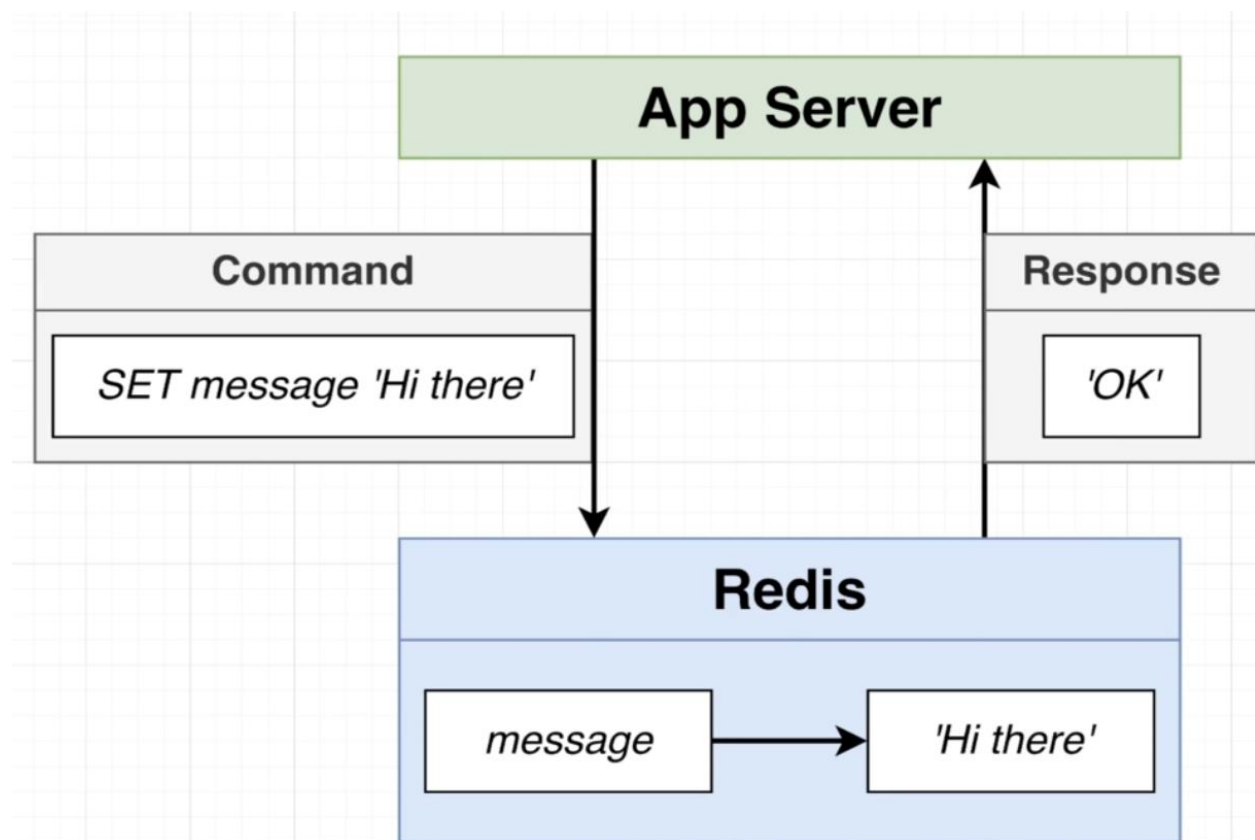
Examples of simple data structure of redis:

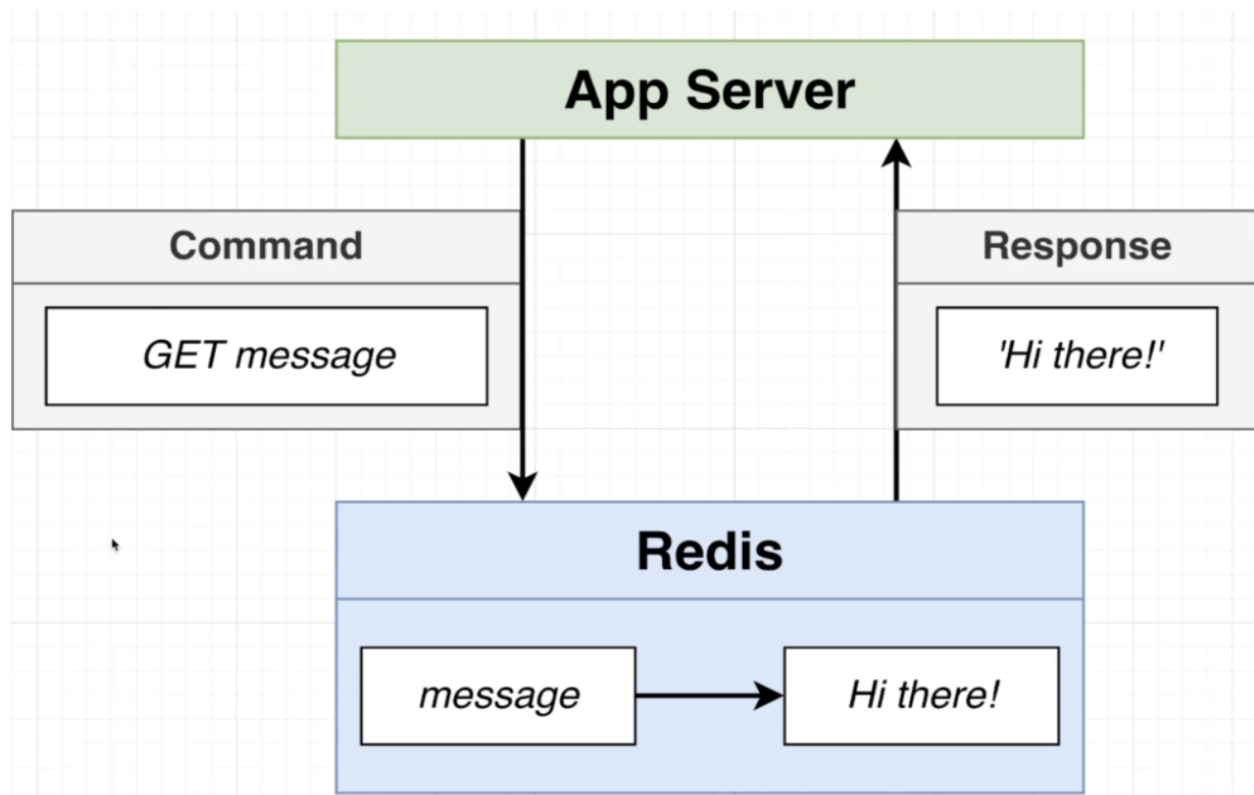


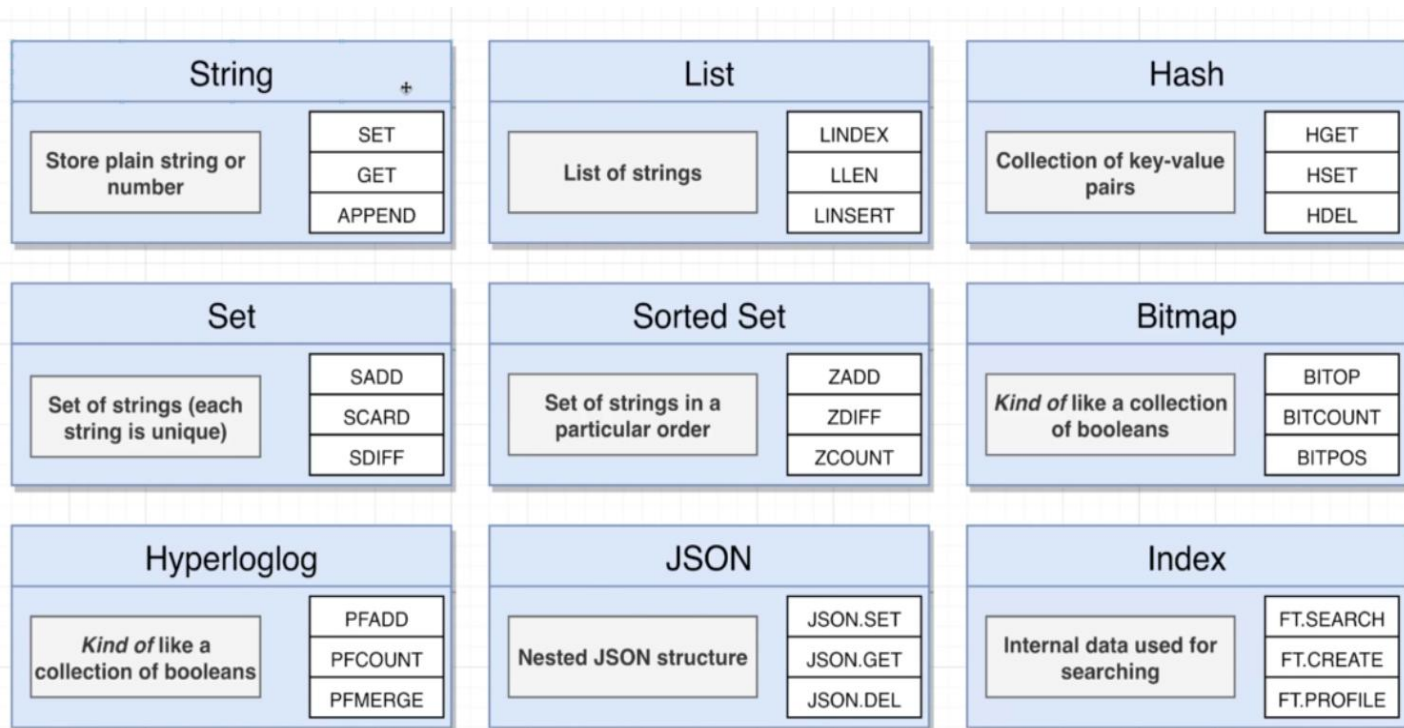
To set basic key, value data in redis, use command: set key value

- Ex1: set jafar loka
- Ex2: set name 'Jafar Loka'

```
127.0.0.1:6379>  
127.0.0.1:6379> set name 'Jafar Loka'  
OK  
127.0.0.1:6379> get name  
"Jafar Loka"  
127.0.0.1:6379> get jafar  
"loka"  
127.0.0.1:6379>
```



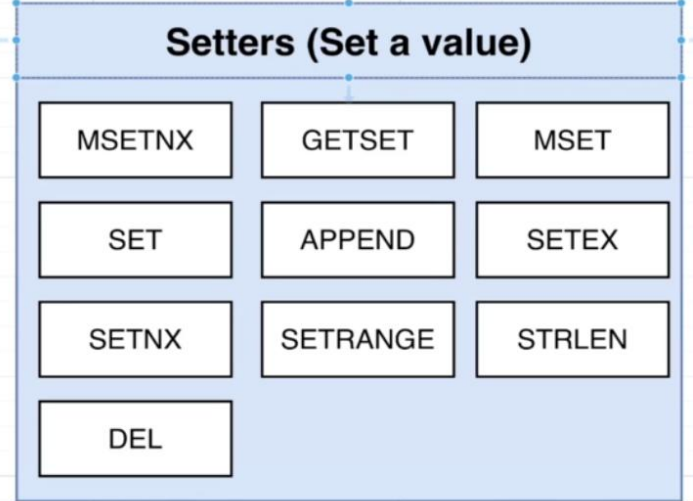
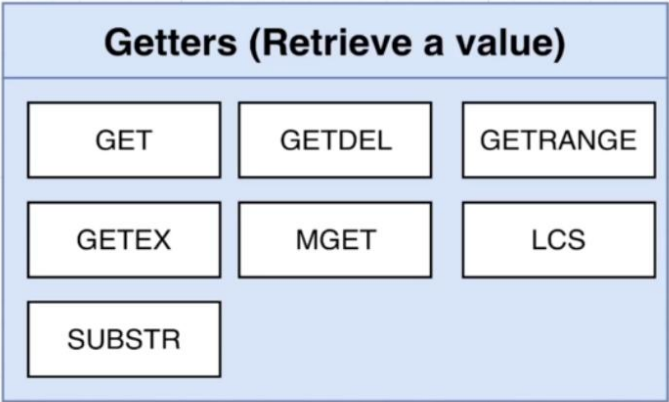


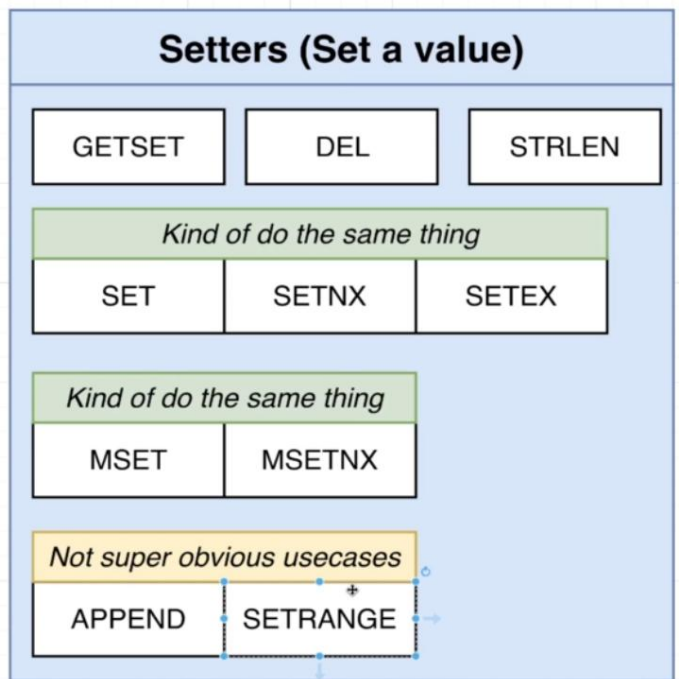
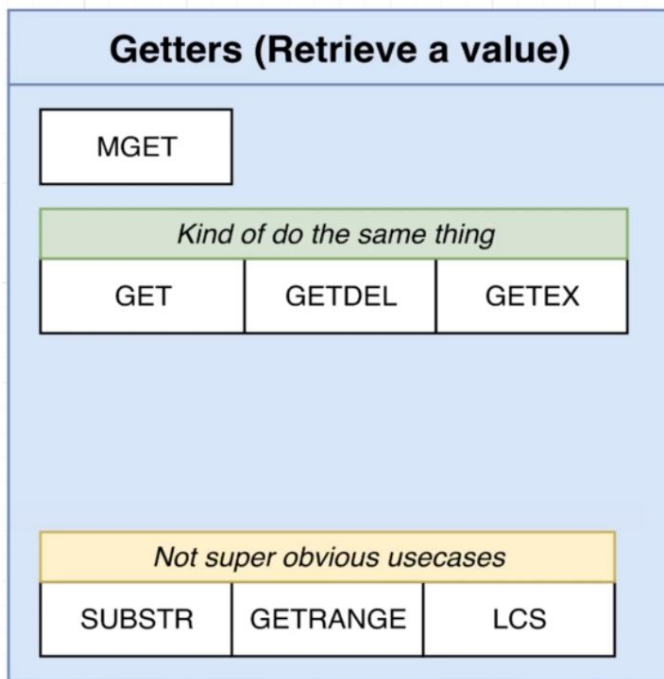


Commands for Handling Strings

APPEND	GET	GETDEL
GETEX	GETRANGE	GETSET
LCS	MGET	MSET
MSETNX	SET	SETEX
SETNX	SETRANGE	STRLEN
SUBSTR	DEL	

*Some of these are also used for
numbers*

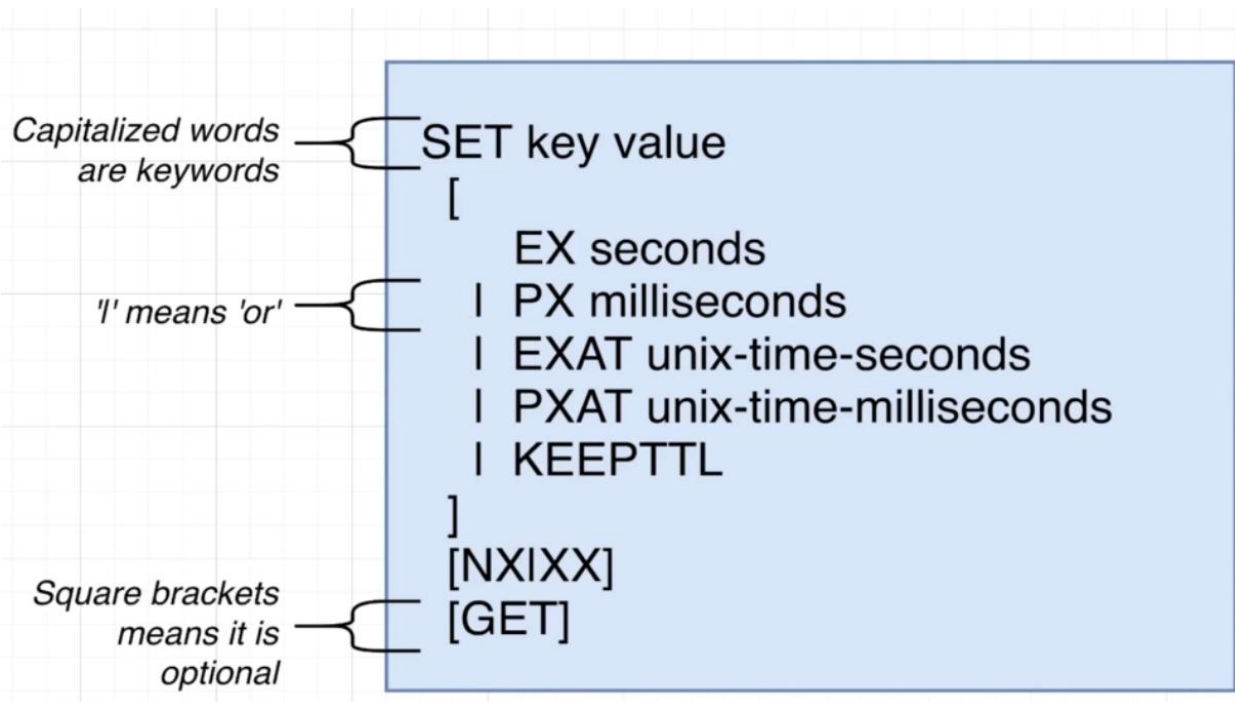




```

127.0.0.1:6379> mset name01 jafar loka 'Jafar Loka'
OK
127.0.0.1:6379> mget name01
1) "jafar"
127.0.0.1:6379> mget name01
1) "jafar"
127.0.0.1:6379> mget loka
1) "Jafar Loka"
127.0.0.1:6379> append name loka
(integer) 4
127.0.0.1:6379> get name
"loka"
127.0.0.1:6379> append name jafar
(integer) 9
127.0.0.1:6379> get name
"lokajafar"

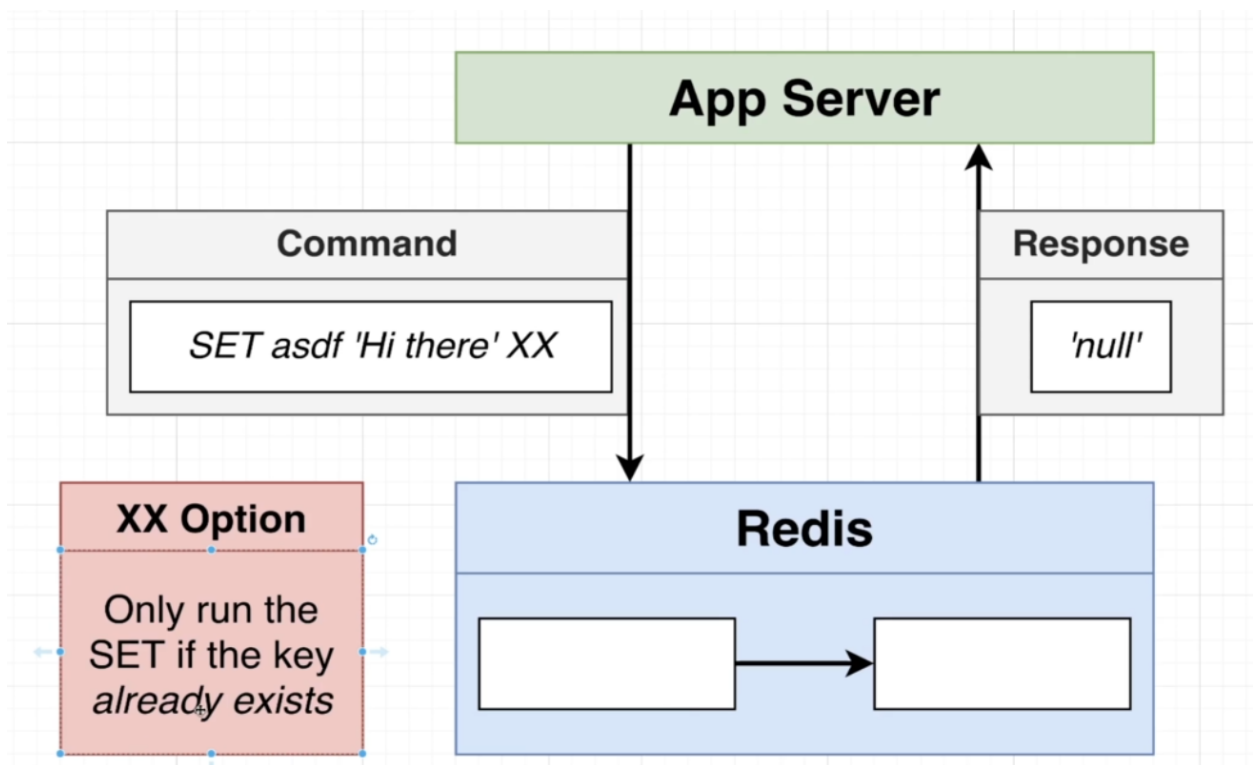
```



To get the previous value of key, when we execute *set* command:

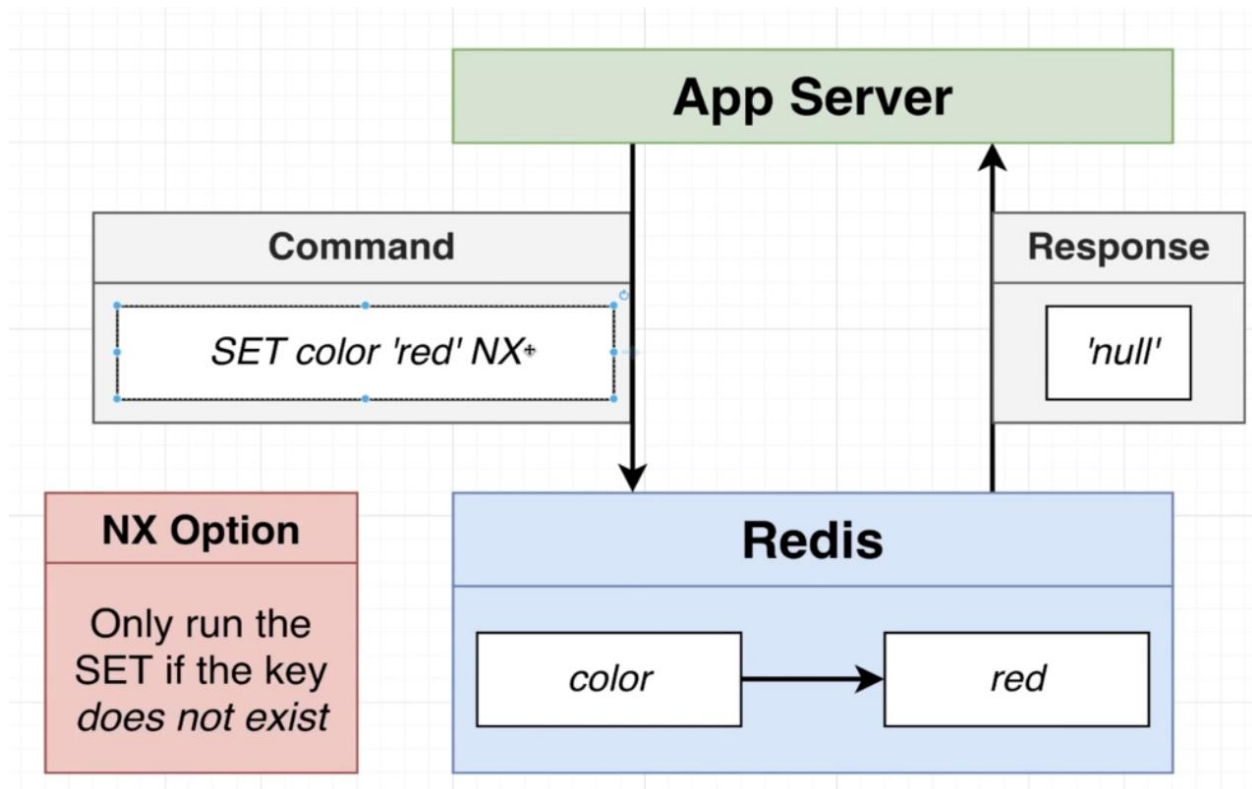
```
127.0.0.1:6379> set name 'JLoka-01' GET
"lokajafar"
127.0.0.1:6379>
```

Only set if key is exists:



```
127.0.0.1:6379> set test test-01 XX GET
(nil)
127.0.0.1:6379> get test
(nil)
127.0.0.1:6379>
```

```
127.0.0.1:6379> set name "Jafar Loka-01" XX GET
"JLoka-01"
127.0.0.1:6379> get name
"Jafar Loka-01"
127.0.0.1:6379>
```



```
127.0.0.1:6379> set test test-01 NX GET
(nil)
127.0.0.1:6379> get test
"test-01"
127.0.0.1:6379> set name 'test-01 && test-02' NX GET
"Jafar Loka-01"
127.0.0.1:6379> get name
"Jafar Loka-01"
127.0.0.1:6379>
```

<https://redis.io/docs/latest/commands/>

To set the TTL value for SET In Seconds we use EX:

```
127.0.0.1:6379> set name jafar-loka-01 EX 1
OK
127.0.0.1:6379> get name
(nil)
127.0.0.1:6379> GET name
(nil)
127.0.0.1:6379> SET name jafar-loka-01 EX 5
OK
127.0.0.1:6379> GET name
"jafar-loka-01"
127.0.0.1:6379> GET name
(nil)
```

To set the TTL for value in milliseconds, we use PX:

```
127.0.0.1:6379> SET name jafar-loka-01 PX 5000 GET
(nil)
127.0.0.1:6379> GET name
"jafar-loka-01"
127.0.0.1:6379> GET name
(nil)
127.0.0.1:6379>
```

To set the TTL for value depending on date and time (unix time seconds):

```
127.0.0.1:6379> SET name jafar-loka-01 EXAT unix-time-seconds [NX|XX] [GET]
```

[Note 1](#): the **SETEX** same as SET with EX.

[Note 2](#): the **SETNX** same as SET with NX.

To add multiple values at once, we use MSET, where M for multiple

```
127.0.0.1:6379> MSET name-01 jafarLoka-01 name-02 JLoka-02 name-03 JLoka-03
OK
127.0.0.1:6379> MGET
(error) ERR wrong number of arguments for 'mget' command
127.0.0.1:6379> MGET name-01 name-02 name-03
1) "jafarLoka-01"
2) "JLoka-02"
3) "JLoka-03"
127.0.0.1:6379> |
```

The syntax for SETEX: *SETEX name 2 jafar-loka-01*

Note: also, we have: **MSETNX**
